UNIVERSITÀ DI TORINO

Hybrid Workflows for Artificial Intelligence

Workflow languages and systems

Prof. Iacopo Colonnelli

iacopo.colonnelli@unito.it

https://alpha.di.unito.it/iacopo-colonnelli

Workflow systems vs task-based libraries

UNIVERSITÀ DI TORINO

Task-based programming libraries focus on **performance** and **productivity**

- They offer a **more productive interface** to parallel/sistributed programming compared to low-level, classical HPC libraries (MPI, OpenMP, OpenACC)
- Their main focus remains **performance**: they trade off ease of use for flexibility and language-specificity

Workflow systems focus on **portability**, **reproducibility**, and **modularity**

- The main goal of a workflow system is to help users design and run **reproducible experiments** that can be conifidently studied by domain experts and extended by other researchers
- Workflow systems **still address large-scale applications**, but they are ready to trade off performance for other non-functional requirements (clarity, modularity, fault tolerance, ...)

The Nextflow WMS

The **Nextflow** workflow system has been one of the first initiatives to promote the usage of **software containers for reproducibility**

While maintaining a specific focus on the **Bioinformatics domain**, nowadays Nextflow is widely used to orchestrate applications in a **diverse set of scientific topics**

The Nextflow curated **workflow catalog**, called **nf-core**, hosts more than 100 pipelines ready to be used and extended by users

P. Di Tommaso, M. Chatzou, E. Floden, P. Prieto Barja, E. Palumbo, and C. Notredame, "Nextflow enables reproducible computational workflows," Nature Biotechnology, 35, 316–319 (2017). doi:10.1038/nbt.3820

UNIVERSITÀ DI TORINO

The Nextflow DSL

Nextflow adopts a **dataflow programming model**, inspired by Kahn process networks, that simplifies writing parallel and distributed pipelines

Workflows are expressed through a **product-specific DSL**, implemented on top of the **Groovy** programming language, which in turn is a super-set of the Java programming language

The Nwextflow runtime leverages on the **Java virtual machine (JVM)** to ensure portability across hardware architectures and operating systems

The Nextflow DSL

In the Nextflow dialect, tasks are called **processes**. Each process specifies:

- The **input data** needed to begin the execution, through the `input` keyword
- The **output data** produced by the taskm through the `output` keyword
- The **shell commands** that must be executed by the task, through the `script` keyword
- Some **additional parameters** (e.g., the folder where output files will be saved through the `publishDir` keyword)

```
process splitString {
    publishDir "results/lower"

    input:
    val x

    output:
    path 'chunk_*'

    script:
    """
    printf '${x}' | split -b 6 - chunk_
    """
}

process convertToUpper {
    publishDir "results/upper"
    tag "$y"

    input:
    path y

    output:
    path 'upper_*'

    script:
    """
    cat $y | tr '[a-z]' '[A-Z]' > upper_${y}
    """
}
```

The Nextflow DSL

Different steps can be concatenated to form a **workflow**, which is specified by the `workflow` keyword

Other than the tasks, a Nextflow workflow defines the **initial parameters** and (optionally) their **default values** through the `params` object

Every interaction between workflow steps reliies on a **channel abstraction**. A channel has two major properties:

- Sending a message is an **asynchronous operation**, i.e., the sender doesn't have to wait for the receiving process
- Receiving a message is a **synchronous operation**, i.e., the receiving process must wait until a message has arrived.

```
params.str = "Hello world!"

workflow {
    ch_str = channel.of(params.str)
    ch_chunks = splitString(ch_str)
    convertToUpper(ch_chunks.flatten())
}
```

Nextflow channels

UNIVERSITÀ DI TORINO

In Nextflow there are two kinds of channels:

- A **queue channel** is a non-blocking unidirectional FIFO queue connecting a producer process (i.e. outputting a value) to a consumer process or an operator
- A **value channel** can be bound (i.e. assigned) with one and only one value, and can be consumed any number of times by a process or an operator

Channel **factories** are functions that can create channels. For example, the `channel.of()` factory can be used to create a channel from an arbitrary list of arguments

Channel **operators** are functions that consume and produce channels, manipulating their values aside from using a process. In practice, operators are useful for implementing the **glue logic between processes**

UNIVERSITÀ
DI TORINO

Execute a Nextflow workflow

Running a Nextflow workflow locally is as simple as typing the following command

```
nextflow run filename.nf
```

However, Nextflow can deploy workflows on a **variety of execution platforms**, including local machines, HPC schedulers, and cloud platforms

Additionally, Nextflow supports a range of **compute environments**, software **container runtimes**, and **package managers**, allowing workflows to be executed in reproducible and isolated environments

Execute a Nextflow workflow

The `process` scope of a Nextflow file allows users to configure the proper exeution options for a given workflow

The `withLabel` and `withName` directives allow to specify **different configurations** for different subsets of processes

```
process {
    executor = 'sge'
    queue = 'long'
    clusterOptions = '-pe smp 10'

    withLabel: big_mem {
        cpus = 16
        memory = 64.GB
        queue = 'long'
    }

    withName: hello {
        cpus = 4
        memory = 8.GB
        queue = 'short'
    }
}
```

Nextflow and containers

Nextflow supports a **variety of container runtimes** to write self-contained and truly reproducible computational pipelines

The same pipeline can be transparently executed with any of the supported container runtimes, depending on which runtimes are available in the target compute environment

The desired container runtime can be configured through a dedicated section in Nextflow DSL file, and different images can be associated with different processes throught the `withLabel` and `withName` directives of the "process' scope

```
process {
    withName:foo {
        container = 'image_name_1'
    }
    withName:bar {
        container = 'image_name_2'
    }
}

apptainer {
  enabled = true
}
```

Common Workflow Language (CWL)

- Open standard to describe analysis **workflows** and **tools**
- Defined through a **scheme**, a **specification** and a conformance **test suite**
- **Portable** and **scalable** on a diverse set of software and execution environments
- Designed to satisfy the requirements of modern **data-intensive pipelines** and to improve workflow **FAIRness**

UNIVERSITÀ DI TORINO

COMMON WORKFLOW LANGUAGE

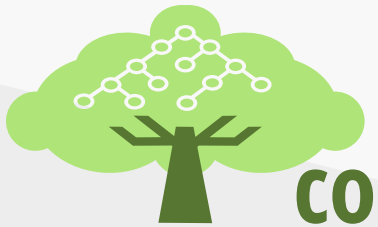
software freedom conservancy

Common Workflow Language (CWL)

The CWL project supports open standards for data analysis based on workflows and tools. In particular, it supports:

- A **pre-standardisation process** during which it is possible to discuss, propose, and test ideas about the standard
- A **standardisation process** that follows the development and release of the standard, according to the **Open Stand** principles
- The **post-standardisation life cycle** management, developing and maintaining training material and courses, tracking potential problems and new proposals from the community

COMMON
WORKFLOW
LANGUAGE

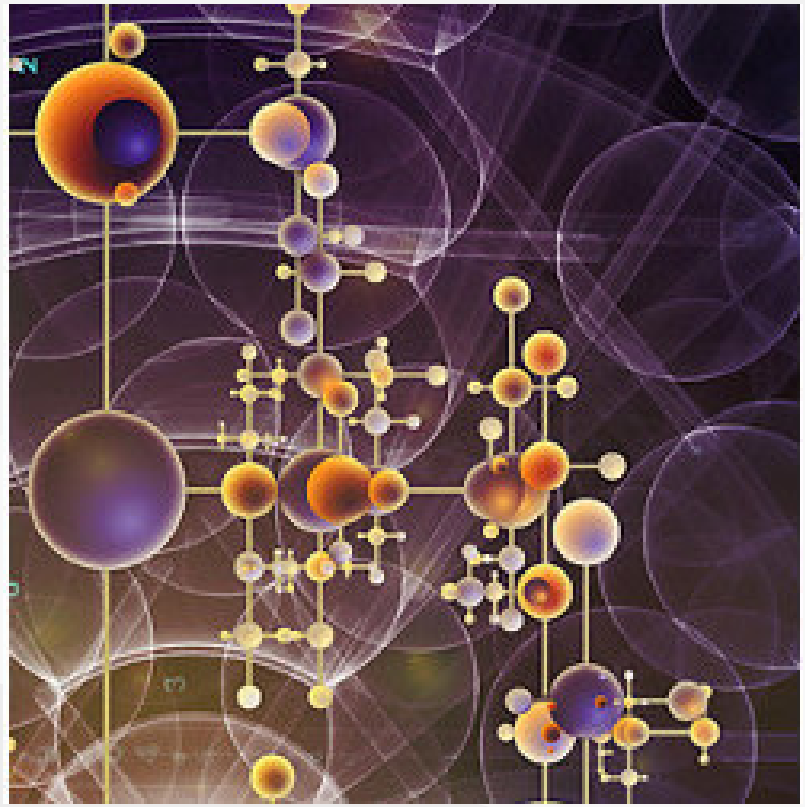


software freedom
conservancy

UNIVERSITÀ
DI TORINO

How to cite CWL

Michael R. Crusoe, Sanne Abeln, Alexandru Iosup, Peter Amstutz, John Chilton, Nebojša Tijanić, Hervé Ménager, Stian Soiland-Reyes, Bogdan Gavrilović, Carole Goble, and The CWL Community. 2022. *Methods Included: Standardizing Computational Reuse and Portability with the Common Workflow Language*. Communications of the ACM 65, 6 (June 2022), 54–63. doi: 10.1145/3486897



UNIVERSITÀ DI TORINO

Why CWL?

- CWL has been designed to be **modular** and **easy to use**
- CWL can export provenance data adhering to **existing standards and ontologies** (e.g., CWLProv or Workflow Run RO-Crate)
- WL exposes **hunderds of no-regression tests** to ensure portability across different versions of the standard
- The declarative nature of CWL allows different optimisations, e.g., **data/location-aware scheduling**, **caching** intermediate results, and **data streaming** between subsequent steps

CWL syntax

CWL collects two different standards: one for the single shell command, and one for workflows

CWL CommandLineTool standard

```yaml
cwlVersion: v1.2
class: CommandLineTool
baseCommand: echo
inputs:
  message_text:
    type: string
    inputBinding:
      position: 1
```

CWL Workflow standard

```yaml
cwlVersion: v1.2
class: Workflow
inputs:
  rna_reads_fruitfly: File
steps:
  quality_control:
    run: bio-cwl-tools/fastqc/fastqc_2.cwl
    in:
      reads_file: rna_reads_fruitfly
    out: [html_file]
outputs:
  quality_report:
    type: File
    outputSource: quality_control/html_file
```

An additional file (written in pure JSON or YAML) specifies the input data for a workflow or a tool

CWL types

CWL is a **strongly-typed** language: each input and output port must specify a type

However, CWL allows **extreme flexibility** in type definitions: it supports **union types**, **optional types**, and the **Any** type

It is also possible to define **complex types** through the **SchemaDefRequirement** directive

```yaml
cwlVersion: v1.2
class: CommandLineTool
requirements:
  SchemaDefRequirement:
    types:
      - $import: biom-convert-table.yaml
inputs:
  table_type:
    type: biom-convert-table.yaml#table_type
    inputBinding:
      prefix: --table-type
...
```

```yaml
#biom-convert-table.yaml
type: enum
name: table_type
label: The type of the table to produce
symbols:
  - OTU table
  - Pathway table
  - Function table
  - Ortholog table
  - Gene table
  - Metabolite table
  - Taxon table
  - Table
```

CWL implementations

Software	Description	Self-Reported Compliance	Platform support
cwltool	Reference implementation of CWL	CWL v1.0 - v1.2	Linux, OS X, Windows, local execution only
Arvados	Distributed computing platform for data analysis on massive data sets. Using CWL on Arvados	CWL v1.0 - v1.2 required 100%	AWS, Azure, Slurm, LSF
Toil	Toil is a workflow engine entirely written in Python.	CWL v1.0 - v1.2	AWS, GCP, Grid Engine, HTCondor, LSF, Slurm, PBS/Torque
StreamFlow	Workflow Management System for hybrid HPC-Cloud infrastructures	CWL v1.0 - v1.2 required 100% (and nearly all optional features)	Kubernetes, HPC with Singularity (PBS, Slurm), Occam, multi-node SSH, local-only (Docker, Singularity)
Calrissian	CWL Engine built for Kubernetes	CWL v1.0 - v1.2 required 100% (and much of the optional features)	Kubernetes
CWL-Airflow	Package to run CWL workflows in Apache-Airflow (supported by BioWardrobe Team, CCHMC)	CWL v1.0 - v1.1	Linux, OS X

CWL in Amazon Omics

Amazon Omics now supports Common Workflow Language

Posted On: Jun 30, 2023

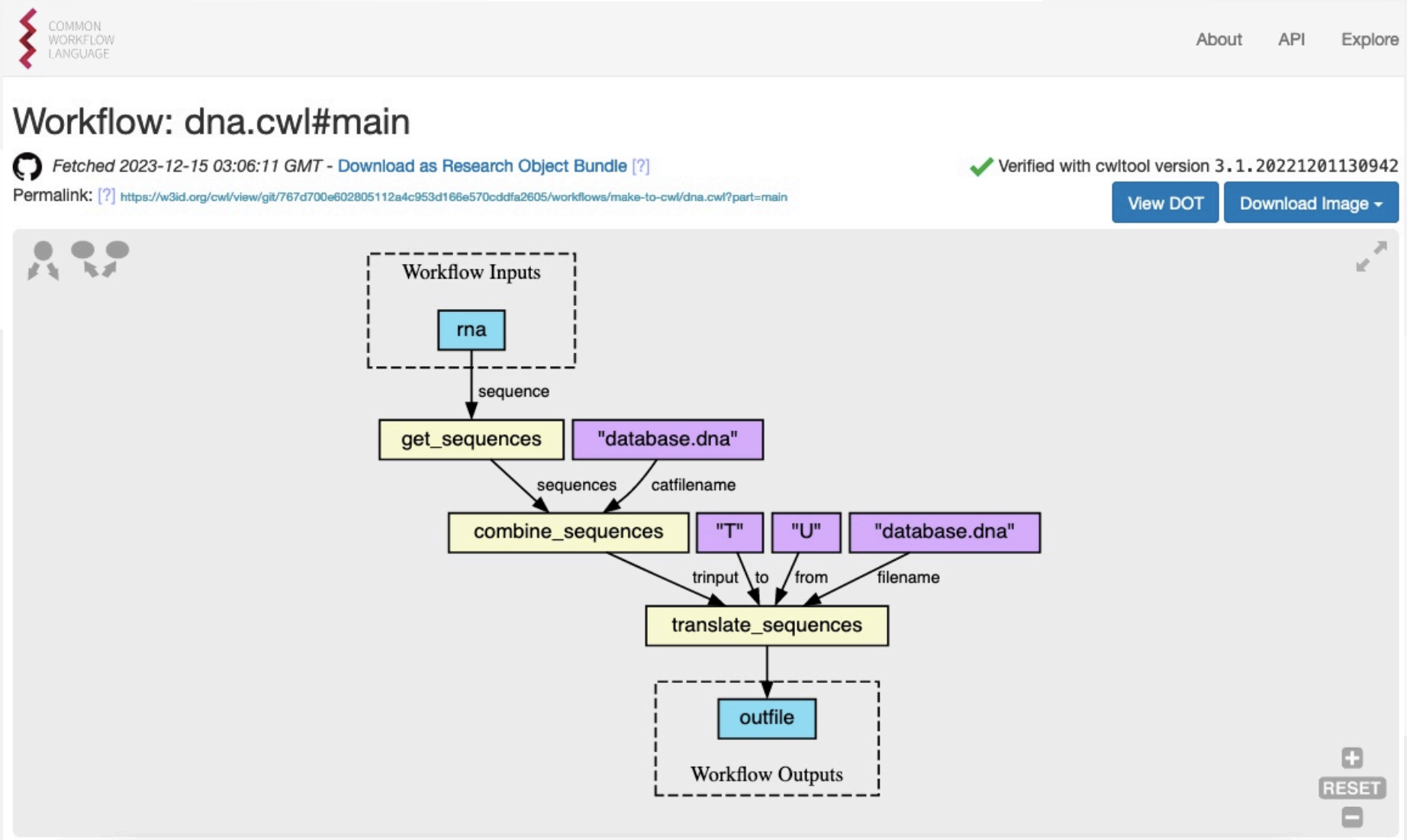
Today, Amazon Omics announces support for Common Workflow Language (CWL) version 1.0-1.2. This new capability extends support for multiple workflow languages (WDL, Nextflow, and CWL) in Amazon Omics, allowing customers to use the workflow language of their choice. Customers can now easily bring their CWL workflows along with their software tools and Amazon Omics will provision and manage all the underlying infrastructure for their workflow runs.

Amazon Omics is a fully managed service that helps healthcare and life science organizations build at-scale to store, query, and analyze genomic, transcriptomic, and other omics data. By removing the undifferentiated heavy lifting, customers can generate deeper insights from omics data to improve health and advance scientific discoveries.

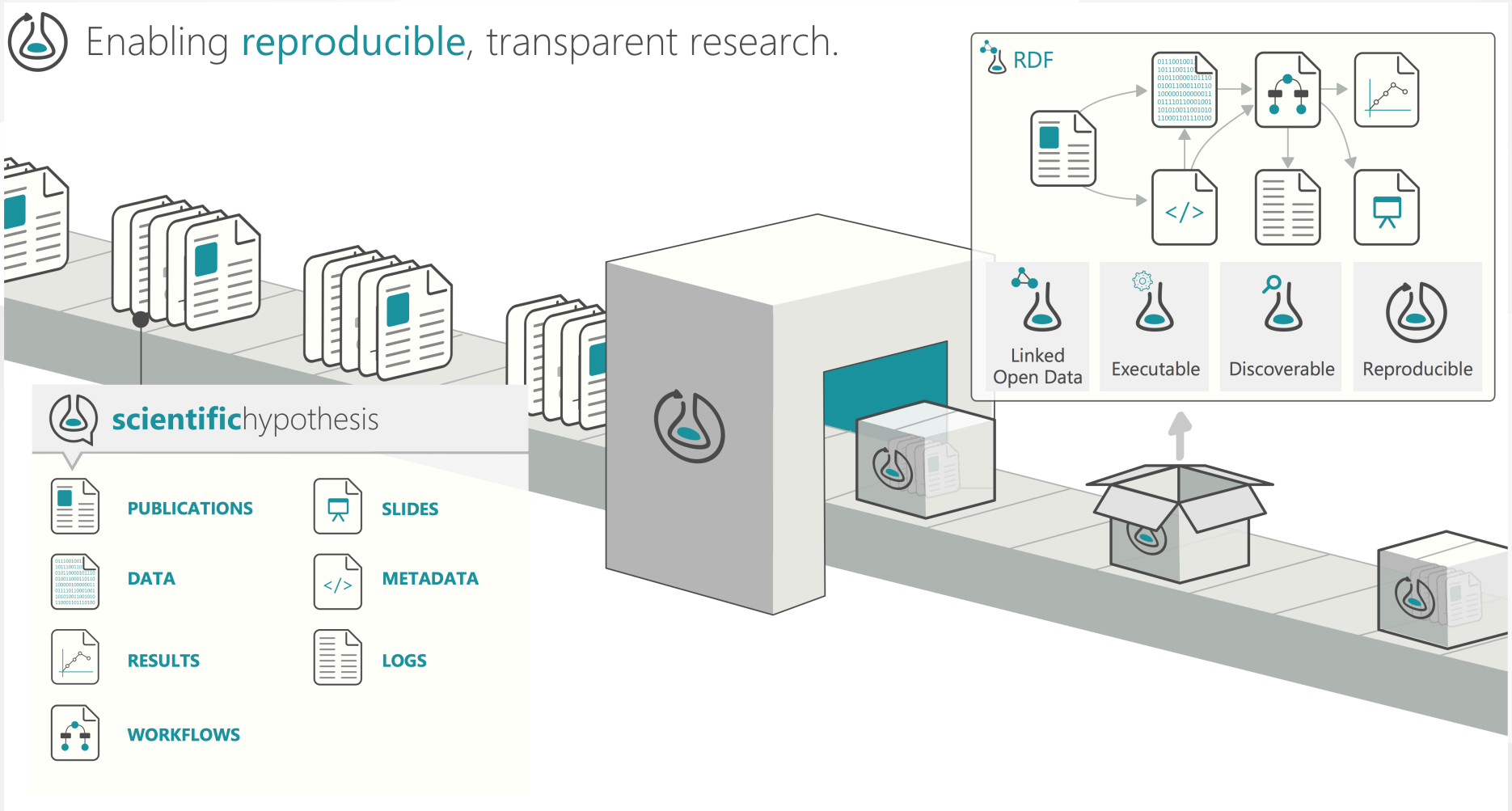
Omics WDL, Nextflow, and CWL workflow languages are available in all AWS Regions where Amazon Omics is generally available: US East (N. Virginia), US West (Oregon), Europe (Frankfurt, Ireland, London), and Asia Pacific (Singapore).

For more information, visit the developer guide.

CWL Ecosystem - CWLViewer



CWL Ecosystem - RO Crate



CWL Ecosystem - WorkflowHub

UNIVERSITÀ DI TORINO

WorkflowHub
A FAIR registry for workflows

SCAN HERE
workflowhub.eu
workflowhub.org

Carole Goble[1], Finn Bacall[1], Stian Soiland-Reyes[1], Stuart Owen[1], Alan Williams[1], Ignacio Eguinoa[2], Bert Droesbeke[2], Hervé Ménager[3], Laura Rodriguez-Navas[4], José M. Fernández[4], Salvador Capella-Gutierrez[4], Michael R. Crusoe[5], Björn Grüning[6], Simone Leo[7], Luca Pireddu[7], Johan Gustafsson[8], Phil Ewels[9], WorkflowHub Community[10], and Frederik Coppens[2]

WorkflowHub is open to any workflows from any system in any discipline.

The Hub is agnostic to Workflow Management Systems. We partner with popular and high profile platforms to co-develop and support metadata mark-up, packaging, registration, repository linkups, search & launching.

Galaxy nextflow COMMON WORKFLOW LANGUAGE jupyter

CWL and containers

CWL supports using **software containers** through the **DockerContainer** requirement

The requirements's format comes from the **Docker** specification. However, each implementation can choose different container runtimes, as lon as they are compatible with Docker format

```yaml
cwlVersion: v1.2
class: CommandLineTool
baseCommand: node
hints:
  DockerRequirement:
    dockerPull: node:slim
inputs:
  src:
    type: File
    inputBinding:
      position: 1
outputs:
  example_out:
    type: stdout
stdout: output.txt
```

CWL and JavaScript

For advanced data manipulation, CWL supports the **JavaScript** language, and more precisely the **EcmaScript6** format

The **ExpressionTool** specification allows to specify a workflow step through a JavaScript program

Since the overhead introduced by this syntax is often significant, use it **only when strictly needed**

```yaml
cwlVersion: v1.2
class: ExpressionTool
requirements:
  InlineJavascriptRequirement: {}
inputs:
  message: string
outputs:
  uppercase_message: string
expression: |
  ${ return {"uppercase_message": inputs.message.toUpperCase()}; }
```

Execute a CWL workflow

The standard way to execute a CWL workflow is through the command line

```
cwl-runner <cwl_file> <input_file>
```

The `cwl-runner` command is a **wrapper** to one of the CWL implementations actually installed

The CWL reference runtime is called **cwltool** and is compatible with all standard features (as it passes all conformance tests). It is implemented in Python

```
pip install cwltool
```

CWL in HPC: scatter/gather

CWL supports **scatter/gather** parallel patterns at the step level since v1.0

If scatter declares more than one input parameter, **scatterMethod** describes how to decompose the input into a discrete set of jobs (dotproduct, nested crossproduct, or flat crossproduct)

```
cwlVersion: v1.2
class: Workflow
requirements:
  ScatterFeatureRequirement: {}
inputs:
  message_array: string[]
steps:
  echo:
    run: hello_workld.cwl
    scatter: message
    in:
      message: message_array
    out: []
outputs: []
```

CWL extensions

The CWL standard allows users to easily implement **extensions** to the base syntax

These extensions are available only in the subset of CWL implementations that decide to support them: they offer **no portability guarantee**

If an extension reaches a high level of maturity and it is of interest for the community, it can be **added to the next version of the standard**

Examples of CWL extensions are the **CUDARequirement**, the **MPIRequirement**, and the **Loop** extension

CWL for HPC: the Loop extension

UNIVERSITÀ DI TORINO

The CWLv1.2 standard can only describe directed **acyclic** graphs, but the vast majority of HPC applications (simulations, deep learning, quantum/classical algorithms) are **iterative**

Solution: we introduced the **cwltool:Loop** extension to describe structured iterative patterns

This extension is currently being implemented in the **CWL v1.3** standard

```yaml
cwlVersion: v1.3.0-dev1
class: Workflow
requirements:
  InlineJavascriptRequirement: {}
inputs:
  i1: int
steps:
  subworkflow:
    when: $(inputs.i1 < 10)
    loop:
      i1: o1
    outputMethod: last
    run: sum.cwl
    scatter: message
    in:
      i1: i1
    out: [o1]
outputs:
  o1:
    type: int
    outputSource: subworkflow/o1
```

CWL4HPC Working Group

UNIVERSITÀ DI TORINO

Goals:

- Improve the CWL **expressive power** with new patterns to better support HPC applications
- Provide **real use cases** of CWL workflows that describe HPC applications
- Create a **virtual community** of CWL users on HPC

Matrix channel: https://matrix.to/#/#cwl4hpc:matrix.org

Exercise

UNIVERSITÀ
DI TORINO

Create a workflow to extract a Java file from a tar archive and compile it

1. Download the `exercise-1.zip` archive and decompress it
2. Create a first step (`CommandLineTool`) to extract the `data/hello.tar` file
3. Create a second step (`CommandLineTool`) that compiles the extracted file `Hello.java` in a `*.class` file using the `openjdk:9.0.1-11-slim` Docker container
4. Merge the two steps in a single CWL `Workflow`
5. Execute the workflow using the `cwl-runner` command

CWL Guide: https://www.commonwl.org/user_guide/topics/yaml-guide.html

Exercise

UNIVERSITÀ
DI TORINO

Translate a simple RNA-Seq pipeline from NextFlow to CWL

1. Download the `exercise-2.zip` archive and decompress it
2. Create a `Docker` image called `$USER/fastqc:0.12.1` and substitute the placeholder inside the `fastqc` step. The FastQC library can be downloaded from https://www.bioinformatics.babraham.ac.uk/projects/fastqc/fastqc_v0.12.1.zip and it requires the `default-jre` and `perl` packages as dependencies
3. Execute the pipelin with NextFlow using the `nextflow run rna-seq.nf` command (https://www.nextflow.io/docs/latest/install.html)
4. Rewrite the pipeline using CWL and execute it using the `cwl-runner` command